5h. In-Place Functions

Martin Alfaro
PhD in Economics

INTRODUCTION

This section continues our exploration of **techniques for mutating vectors**. The emphasis is now on **in-place functions**, defined as functions that mutate at least one of their arguments.

Many built-in functions in Julia have an in-place counterpart, which can be easily recognized by the `!` suffix in their names. These versions store the output in one of the function arguments, thereby avoiding the creation of a new object. In practice, it means that one or more of the arguments will be updated immediately after executing the function. For example, given a vector `x`, the call `sort(x)` produces a new vector with ordered elements, but without altering the original `x`. In contrast, the in-place version `sort!(x)` overwrites the content of `x`.

> **Appending ! To A Function Has No Impact on the Code**
>
> **Appending `!` to a function's name doesn't change the function's behavior**. It simply signals to users that the function performs a mutating operation. The goal is to make side effects explicit, helping programmers avoid unintended modifications of objects.

The benefits of in-place functions will become evident in Part II, when discussing high-performance computing. Essentially, by reusing existing objects, in-place functions eliminate the memory overhead associated with creating new objects.

DEFINING IN-PLACE FUNCTIONS

In-place functions, also known as **mutating functions**, are characterized by their ability to modify at least one of their arguments. For example, all the following are considered in-place functions. Following the convention in Julia, we denote them by appending `!` to the function's name.

IN-PLACE FUNCTION

```
x = [1,1]


function foo!(x)
    x[1] = 0
end
```

```
julia> x
2-element Vector{Int64}:
 1
 1
julia> foo!(x) #it mutates 'x'
julia> x
2-element Vector{Int64}:
 0
 1
```

IN-PLACE FUNCTION

```
y = [1,1]


function foo!(x)
    x[1] = 0
end
```

```
julia> y
2-element Vector{Int64}:
 1
 1
julia> foo!(y) #it mutates 'y'
julia> y
2-element Vector{Int64}:
 0
 1
```

IN-PLACE FUNCTION

```
x = [1,1]
y = [2,2]

function foo!(x,y)
    x[1] = 0
end
```

```
julia> x
2-element Vector{Int64}:
 1
 1
julia> foo!(x,y) #it mutates one of the arguments (just x)
julia> x
2-element Vector{Int64}:
 0
 1
```

Functions Can't Reassign Variables

While functions are capable of mutating values, they **can't reassign variables defined outside their scope**. Any attempt to redefine such variable will be interpreted as the creation of a new local variable.

The following code illustrates this behavior by redefining a function argument and a global variable. The output reflects that `foo` in each example treats the redefined `x` as a new local variable, only existing within `foo`'s scope.

FUNCTION ARGUMENT X

```
x = 2

function foo(x)
    x = 3
end
```

```
julia> x
2
julia> foo(x)
julia> x #functions can't redefine global variables, only mutate them
2
```

GLOBAL VARIABLE X

```
x = [1,2]

function foo()
    x = [0,0]
end
```

```
julia> x
2-element Vector{Int64}:
 1
 2
julia> foo()
julia> x #functions can't redefine variables globally, only mutate them
2-element Vector{Int64}:
 1
 2
```

Strictly speaking, it's possible to reassign a variable by denoting a variable with the `global` keyword. However, its use is typically discouraged, explaining why we won't cover it.

BUILT-IN IN-PLACE FUNCTIONS

In Julia, many built-in functions that operate on vectors are available in two forms: a standard version that returns a new object and an in-place version that mutates its argument. To distinguish them, Julia's developers follow the naming convention that **any function ending with `!` corresponds to an in-place function**.

An example is given by the functions `sort` and `sort!`. Both arrange the elements of a vector in ascending order, with the option `rev=true` implementing a descending order. In its standard form, `sort(x)` creates a new vector containing `x`'s elements sorted, but leaving the original vector `x` unchanged. In contrast, the in-place version `sort!(x)` directly updates the original vector `x`, overwriting its contents with the sorted values. Both functions perform the same conceptual task, but they differ in whether they allocate new memory or reuse existing storage.

SORT - STANDARD VERSION

```
x      = [2, 1, 3]

output = sort(x)
```

```
julia> x
3-element Vector{Int64}:
 2
 1
 3
julia> output
3-element Vector{Int64}:
 1
 2
 3
```

SORT - IN-PLACE VERSION

```
x      = [2, 1, 3]

sort!(x)
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3
```

It's also common to have in-place functions that accepts arguments only serving as a destination for their output. This argument isn't involved in the computation itself. Instead, it provides pre-allocated storage for the function to write its results. By doing so, we avoid allocating a new array every time the function runs.

For instance, `map(foo, x)` applies the function `foo` to each element of `x` and returns a freshly allocated vector. In contrast, `map!(foo, output, x)` performs the same element-wise transformation, but writes directly into the existing vector `output`.

MAP

```
x      = [1, 2, 3]


output = map(a -> a^2, x)
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3
julia> output
3-element Vector{Int64}:
 1
 4
 9
```

MAP!

```
x      = [1, 2, 3]
output = similar(x)             # we initialize `output`

map!(a -> a^2, output, x)       # we update `output`
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3
julia> output
3-element Vector{Int64}:
 1
 4
 9
```

FOR-LOOP MUTATIONS VIA IN-PLACE FUNCTIONS

In Julia, any performance-critical code should live inside functions. This not only prevents issues with variable scope, but it's also key for performance. The subject will be discussed extensively in Part II of the book. Similarly, for-loops often provide the most direct path to high performance in Julia.

When both aspects are combined, the ability of functions to mutate their arguments via for-loops enables the reuse of existing storage, instead of allocating new arrays repeatedly.

A typical strategy to implement these operations is to initialize vectors with `undef` values, pass them to a function, and fill them element by element via a for-loop. The examples below illustrate this approach.

FOR-LOOP MUTATION

```
x = [3,4,5]

function foo!(x)
    for i in 1:2
        x[i] = 0
    end
end
```

```
julia> foo!(x)

julia> x
3-element Vector{Int64}:
 0
 0
 5
```

POPULATING VECTOR WITH FOR-LOOP

```
x = Vector{Int64}(undef, 3)          # initialize a vector with 3 elements

function foo!(x)
    for i in eachindex(x)
        x[i] = 0
    end
end
```

```
julia> foo!(x)

julia> x
3-element Vector{Int64}:
 0
 0
 0
```